

CryptoChain Analyzer — Technical Report

Student: Victor Valdivia Calatrava

Course: Cryptography — Universidad Alfonso X el Sabio

Date: May 2026

1. Cryptographic Metrics Implemented

M1 — Proof of Work Monitor

The module displays live Bitcoin mining statistics derived from the last N confirmed blocks (20, 50, or 100, configurable in the sidebar) fetched from the Blockstream API. The central cryptographic concept is the **target**: a 256-bit integer that every valid block hash must be numerically below.

Bitcoin stores the current target in compact form as the `bits` field of the block header — a 4-byte little-endian integer that encodes an exponent and a 3-byte coefficient: $\text{target} = \text{coefficient} \times 2^{(8 \times (\text{exponent} - 3))}$. The module decodes this field manually using only Python's `struct` and integer arithmetic, without any Bitcoin-specific library, producing the full 256-bit target. Mining difficulty is then defined as the ratio of the genesis-block target (`0x00000000FFFF0000...0000`) to the current target: a higher difficulty means a smaller target and therefore harder work.

The module also plots the **distribution of inter-arrival times** (seconds between consecutive blocks) overlaid with the theoretical Exponential PDF. This distribution arises from the memoryless nature of the mining process: each double-SHA256 attempt independently succeeds with probability $p \approx \text{target} / 2^{32}$. Because each attempt is independent (the Markov property), the geometric waiting-time distribution converges in the continuous-time limit to an Exponential with rate $\lambda = 1 / 600 \text{ s}$ — analogous to radioactive decay or Poisson arrival processes. The module fits $\lambda = 1 / \text{mean}(\text{observed gaps})$ by Maximum Likelihood Estimation and overlays the resulting PDF on the histogram. Network hash rate is estimated as $\text{difficulty} \times 2^{32} / 600$, since by convention $\text{difficulty} = 1$ requires 2^{32} expected hashes.

M2 — Block Header Analyzer

The 80-byte Bitcoin block header is parsed field by field using `struct.unpack` with little-endian byte order: version (4 bytes at offset 0), previous block hash (32 bytes), Merkle root (32 bytes), timestamp (4 bytes at offset 68), bits (4 bytes at offset 72), and nonce (4 bytes at offset 76). The 32-byte hash fields are stored in internal (little-endian) byte order on disk, but are displayed byte-reversed following the big-endian / RPC convention inherited from Satoshi's original client.

The module then **verifies the Proof of Work locally** using only `hashlib.sha256`. It computes `SHA256(SHA256(raw_80_bytes))`, reverses the 32-byte result to obtain the display hash, and compares it to the hash returned by the API. It also counts the leading zero bits in the display-order hash and checks that the hash integer (in internal byte order) is strictly less than the target decoded from `bits`. The entire verification is performed without trusting any Bitcoin library — a direct implementation of the cryptographic specification.

M3 — Difficulty History

Every 2016 blocks (approximately two weeks at 10 min / block), Bitcoin recalculates the mining target using Satoshi's Difficulty Adjustment Algorithm: $\text{new_difficulty} = \text{old_difficulty} \times (2016 \times 600 \text{ s} / \text{actual_elapsed})$. An anti-manipulation clamping rule prevents any single adjustment from exceeding a factor of 4× or falling below 0.25×, guarding against sudden hash-rate spikes or crashes.

The module fetches two years of difficulty history from the mempool.space API (/v1/mining/hashrate/2y) and plots the difficulty time series annotated with vertical markers at each 2016-block epoch boundary. A secondary bar chart displays the actual/expected block-time ratio per period, derived by inverting the DAA formula: $\text{ratio} = \text{old_difficulty} / \text{new_difficulty}$. Bars above 1.0 indicate slower-than-expected blocks (difficulty dropped next period); bars below 1.0 indicate faster blocks (difficulty rose).

M5 — Merkle Proof Verifier

A Merkle tree (Merkle, 1979) is a binary hash tree in which each leaf is a transaction ID (txid) and each internal node is the double-SHA256 of its two children concatenated. The single 32-byte root stored in the block header commits to every transaction in the block.

The module builds the complete Merkle tree from scratch using only `hashlib`. Two implementation details are critical: (1) txids from the Blockstream API are in display (big-endian) order and must be byte-reversed to internal order before hashing; (2) if any level has an odd number of nodes, the last node is duplicated before pairing — a quirk of Bitcoin Core's original implementation (`src/consensus/merkle.cpp`). The computed root is compared byte-for-byte against the `merkle_root` field parsed from the raw block header.

A **Merkle inclusion proof** for transaction i consists of only $\lceil \log_2(N) \rceil$ sibling hashes, one per tree level. Verification re-derives the root by iteratively hashing the target node with each sibling (concatenation order determined by left/right side); if the result equals the header root, the transaction is cryptographically proven to be in the block without trusting the API. For a typical block with $N = 3\,000$ transactions, the proof requires 12 hashes (384 bytes) versus $3\,000 \times 32 = 96\,000$ bytes for the full txid list.

M6 — Security Score: Cost of a 51% Attack

A 51% attack requires controlling more than half the network hash rate, enabling an attacker to produce a secret longer chain and double-spend transactions. The module estimates the real-time cost in USD/hour of supplying 51% of the current hash rate, using the **Antminer S19 XP** as the reference hardware unit (140 TH/s, 3 010 W, ~\$2 000 on the secondary market in 2024). The electricity price defaults to \$0.05/kWh (industrial mining facility rate) and is adjustable via an interactive slider. Electricity cost is computed as $n_{\text{asics}} \times 3.01 \text{ kW} \times \text{price_per_kWh}$, and the capital cost is amortized over a 3-year hardware lifetime (26 280 hours).

The **attack success probability** is computed from Nakamoto's exact §11 formula. While the honest chain gains z confirmation blocks, the attacker mines $k \sim \text{Poisson}(\lambda = z \times q/p)$ blocks in secret. The probability of ever catching up (Gambler's ruin) is:

$$P_{\text{success}}(z) = 1 - \sum_{k=0}^z \frac{e^{-\lambda} \lambda^k}{k!} \cdot \left(1 - \left(\frac{q}{p}\right)^{z-k}\right)$$

For $q < 0.5$, this probability decreases exponentially with z . The module plots curves for $q = 10\%$, 20% , and 30% , marking the conventional 6-confirmation threshold. At $q = 10\%$, six confirmations reduce the success probability to less than 0.1%.

2. AI Components

M4 — Block Anomaly Detector

Model and justification. M4 is an unsupervised statistical anomaly detector based on the Exponential distribution. No supervised learning model was appropriate here because no ground-truth labels exist for "anomalous" blocks. The Exponential model is the principled choice: it is the exact theoretical distribution of Bitcoin inter-arrival times under the Poisson process assumption, and its single parameter λ is estimated from the data itself without hyperparameter tuning.

Training data. The model operates on inter-arrival times (seconds between consecutive blocks) derived from the last 200 confirmed blocks fetched from Blockstream. No historical dataset is stored; the model is re-fitted to live data on each page load.

Algorithm. The rate parameter $\lambda = 1 / \text{mean}(\text{observed gaps})$ is estimated by MLE. For each observation t , the survival probability $p_value = \exp(-\lambda t)$ gives the probability of seeing a gap of at least t seconds under the null model. A block is flagged as anomalous if $p_value < 0.01$ (statistically rare long gap) or $t < 60$ s (near-simultaneous block find, common in large mining pools).

Evaluation metrics and results. Because no ground-truth labels exist, the Kolmogorov-Smirnov test is used to evaluate the distributional assumption underlying the anomaly criterion — it is the most appropriate formal metric for this setting. Results from a representative run on the last 200 blocks:

Metric	Value	Interpretation
Fitted mean gap (λ^{-1})	669.8 s	MLE estimate; expected 600 s
Anomalies detected	26 / 199 (13.1%)	Blocks flagged as statistically rare
KS statistic	0.0416	Near zero — good distributional fit
KS p-value	0.8666	> 0.05: data consistent with $\text{Exp}(\lambda)$

The KS p-value of 0.8666 confirms that inter-arrival times are consistent with the exponential null model. The fitted mean of 669.8 s (slightly above 600 s) reflects a momentarily slower network at the time of sampling.

Limitations. The model assumes constant hash rate — violated during difficulty transitions or pool coordination events. The 200-block observation window covers approximately 33 hours, which may not capture longer-term trends. The observed anomaly rate (13.1%) exceeds the theoretical rate under the null model (~2%), indicating moderate hash-rate variability in this sample rather than adversarial behavior.

M7 — Transaction Fee Estimator

Model and justification. M7 is a supervised regression model predicting the next period's median Bitcoin transaction fee (sat/vByte). A **Gradient Boosting Regressor** (GBR, scikit-learn) was chosen because Bitcoin fees exhibit non-linear interactions: intraday usage peaks, day-of-week cycles, and autocorrelation from congestion episodes. Linear regression was rejected because the fee distribution is right-skewed and the interactions between lag features and time-of-day cannot be modelled linearly. An LSTM was rejected as

unnecessarily complex for a proof-of-concept on ~800 samples. GBR handles these patterns without linearity assumptions, is interpretable through feature importances, and generalises well without hyperparameter tuning at this sample size.

Training data. Historical per-block median fee rates (`avgFee_50`, sat/vByte) are fetched from mempool.space (`/v1/mining/blocks/fee-rates/1w`), covering approximately the last week of confirmed blocks. Six features are engineered from each observation: `hour_of_day` (intraday cycle), `day_of_week` (weekend vs. weekday demand), `lag_1` (previous period's median fee — strongest autocorrelation), `lag_3`, `lag_6` (short-term trend), and `rolling_std_6` (6-period rolling standard deviation, capturing fee volatility). The target variable is the **next** period's median fee (1-step-ahead prediction). The dataset is split chronologically 80% / 20% without shuffling, ensuring the test set always consists of the most recent samples and preventing look-ahead bias. GBR hyperparameters: 200 estimators, `max_depth` = 4, `learning_rate` = 0.05, `subsample` = 0.80, `random_state` = 42.

Results.

Metric	Value
Training samples	634
Test samples	159
MAE	3 sat/vByte
RMSE	1.13 sat/vByte
R ²	0.014

The low R² reflects a limitation of the 1-week training window: if fee rates are relatively stable or punctuated by sudden spikes (e.g., Ordinals inscription bursts) not well represented in recent history, the model converges toward the mean. This is a data availability constraint: mempool state (pending transaction count, block space utilization) would be the most informative additional feature but was not accessible via the available public API endpoints.

3. References

Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System*. <https://bitcoin.org/bitcoin.pdf>

Blockstream API Documentation. *Blockstream Explorer API*. <https://blockstream.info/api>

Mempool.space API Documentation. *Mempool.space API*. <https://mempool.space/api>

Merkle, R. C. (1979). *Secrecy, Authentication, and Public Key Systems*. Stanford University PhD Dissertation. (Original description of hash trees used in Bitcoin's Merkle tree construction.)

Bitcoin Core contributors. *src/consensus/merkle.cpp*. <https://github.com/bitcoin/bitcoin> (Implementation of the Merkle tree duplication rule referenced in M5.)